

Project 1: Content-based Recommendation

추천시스템 · 2025-27567 배문성 · 2026년 9월

1. 전체 구조

main.py는 데이터 적재(load_game_data, load_ratings_data) → representation 생성(build_semantic_matrix, build_tfidf_matrix) → 추천·예측(ItemCF, WeightedItemCF) → 출력(write_output) 순서로 동작한다. representation만 교체하면 동일한 로직을 쓸 수 있으므로 Item-based CF는 ItemCF 클래스 하나로 구현하고 semantics용·TF-IDF용 인스턴스를 각각 생성했다. ratings.csv의 game_id는 원본 ID(예: 282440)이므로 build_game_index로 game_id → 행렬 row index 매핑을 만들어, 행렬에 접근하기 전에 반드시 이 매핑을 거치도록 했다.

2. Description 기반 representation (Task 1)

- load_game_data(): games_metadata.json을 한 줄씩 json.loads로 파싱한 뒤 game_id 기준으로 정렬하여 game_ids, descriptions, tags를 같은 순서로 반환한다. 이후 만드는 모든 행렬의 행 순서가 이 정렬 순서와 일치한다.
- build_semantic_matrix(): SentenceTransformer("all-MiniLM-L6-v2")로 description을 일괄 인코딩해 shape [19755, 384] 행렬을 만든다. CPU 인코딩에 약 1분이 걸리므로 결과를 cache/semantic_matrix.npz에 저장하고, 모델 이름과 description 전체의 해시가 같을 때만 재사용한다.

3. Tag 기반 TF-IDF representation (Task 2)

build_tfidf_matrix()에서 전체 tag를 모아 alphabetical order로 정렬해 열 순서를 고정한다(unique tag 440개). 이후

- IDF**: Counter로 각 tag가 등장한 game 수를 세고 $\log_{10}(\text{total_count} / \text{tag_count})$ 로 계산.
- TF**: game 하나에 대해 $n(d, t) / n(d)$, 즉 해당 tag의 등록 횟수를 그 game의 전체 tag 수로 나눈 값.
- 각 game이 실제로 가진 tag만 순회하며 $\text{tfidf_matrix}[\text{row}, \text{tag_col}[\text{tag}]] += \text{idf}[\text{tag}] / n_d$ 로 누적해 shape [19755, 440] 행렬을 완성한다. tag가 없는 game은 0 벡터로 남는다.

4. Cosine similarity (Task 3)

ItemCF.__init__에서 representation을 행 단위로 L2 정규화한다($V / \max(\|V\|, \epsilon)$). 이렇게 두면 이후의 모든 내적이 곧 cosine similarity가 되므로, [19755, 19755] 유사도 행렬(약 1.5 GiB)을 실제로 만들지 않고도 필요한 유사도를 그때그때 얻

을 수 있다. 분모의 ϵ 은 0 벡터(tag가 없는 game)로 인한 0 나눗셈을 막는다.

5. 예측 평점 계산 (Task 4)

5.1 user 이력 추출 (4-a)

UserHistory는 user 815,006명 × game 19,755개 행렬(약 103 GiB, 99.99%가 빈 칸)을 만들지 않는다. 대신 (row, rating)을 user_id로 정렬한 테이블 하나로 보관하고, 요청받은 user의 구간만 꺼내 쓴다(get()). metadata에 없는 game_id는 representation이 없으므로 이 단계에서 제외한다.

5.2 Content 기반 Item-based CF (4-b)

ItemCF.predict_all(user_id)는 해당 user가 전체 game에 남길 것으로 추정되는 점수(shape [19755])를 다음과 같이 계산한다.

$$\hat{r}(u, i) = \frac{\sum_{j \in I_u} r_{uj} \cdot \text{sim}(j, i)}{\sum_{j \in I_u} \text{sim}(j, i) + \epsilon}$$

분자·분모 모두 sim에 대해 선형이므로, [n, 19755] 유사도 행렬을 만들지 않고 $v @ (\text{ratings} @ V[\text{rows}])$ 와 $v @ V[\text{rows}].\text{sum}(\text{axis}=0)$ 처럼 벡터 하나로 접을 수 있다. 근사가 아니라 수학적으로 동일한 값이며, user당 행렬곱 두 번으로 끝난다.

recommend()는 이미 interact한 game을 후보에서 제외한 뒤(np.setdiff1d), 소수점 4자리로 반올림한 점수 내림차순, 동점이면 game_id 오름차순으로 상위 20개를 고른다(np.lexsort). 이력이 1개인 user는 모든 후보 점수가 $rs / (s + \epsilon) \approx r$ 이 되어 10^{-8} 수준의 차이만 남으므로, 반올림 전 점수로 정렬하면 부동소수점 오차가 동점 순서를 정한다. 실제로 TF-IDF 추천 400줄 중 약 절반이 float32/float64 선택에 따라 달라졌고, 반올림 후 정렬에서는 모두 같았다. 유사도 합이 0 근처이면 오차가 증폭되므로 정규화와 누적은 float64로 계산한다. 평가 이력이 없는 user에 대해서는 0 벡터를 반환한다.

5.3 가중합과 대량 쌍 예측 (4-c)

WeightedItemCF는 두 ItemCF의 예측을 $\text{Score} = \alpha \cdot \text{Score}_{\text{sem}} + (1 - \alpha) \cdot \text{Score}_{\text{tfidf}}$ 로 합친다. 명세가 조절을 허용한 것은 α 뿐이므로 clip이나 평균 대체 같은 후처리는 하지 않는다. 따라서 평가 이력이 없는 user는 두 모델 모두 0을 내어 최종 예측도 0이고, 유사도 합이 0에 가까우면 예측이 [1, 5]를 크게 벗어날 수 있다.

predict_pairs()는 4-c 출력과 검증(ratings.csv 약 106만 쌍 등)에 공통으로 쓰는 대량 쌍 예측 경로다. predict()를 반복하면 쌍마다 [19755, d] 행렬곱이 필요해 사실상 끝나지 않으므로, 쌍마다 그 user의 이력만 펼쳐 np.einsum으로 내적하고 np.bincount로 쌍 단위로 되합친다(연산량 $\sum_u n_u^2 \approx 3.1 \text{ M}$). searchsorted로 각 쌍의 user 구간을 한 번에 찾고, 메모리를

고려해 펼친 이력이 50,000행을 넘지 않도록 쌍을 나눠 처리한다. 값은 `predict()`를 반복한 것과 동일하다. `score_rmse()`는 이를 이용해 실제 rating과의 RMSE를 계산한다.

5.4 가중치 α 최적값

최적값은 $\alpha = 0.08$ 이다. `test`는 `ratings.csv`에서 제외된 interaction이므로 검증도 interaction 단위로 나누고, 매번 train 평점만으로 `UserHistory`를 다시 만들어 평가 대상 평점이 예측에 섞이지 않게 했다(`validate.py`, `seed 0`).

1. 전체의 10%를 holdout으로 떼어 두고, 나머지 90%를 5-fold로 나눈다.
2. fold마다 두 모델 점수를 한 번만 구하고 `combine()`으로 $\alpha \in \{0.01, \dots, 0.99\}$ 를 모두 평가한다.
3. 5개 fold의 제곱오차를 합쳐 전체 RMSE가 최소인 α 를 고르고, holdout에서 0.5와 비교한다.

RMSE	$\alpha = 0.5$	$\alpha = 0.08$
5-fold CV	3.1235	3.1034
5-fold CV (이력 있는 user)	1.5021	1.3556
holdout	3.0448	3.0375
holdout (이력 있는 user)	1.3873	1.3387

검증쌍의 약 70%는 train 이력이 없는 user라 α 와 무관하게 0으로 예측되고(RMSE 약 3.6), 이 오차가 전체 RMSE를 지배한다. 그래서 전체 RMSE는 $\alpha \in [0.01, 0.15]$ 에서 거의 평탄하고 fold별 최적값도 0.03–0.33으로 흔들렸다(holdout 자체 최적값 0.15). holdout 쌍 단위 paired bootstrap(1,000회)에서 MSE 차이의 95% 구간은 $[-0.119, 0.011]$ 로 0을 포함했는데, 유사도 합이 0에 가까운 소수의 semantic 이상치(오차 최대 242)가 분산을 키우기 때문이다. 반면 이력 있는 user만 보면 α 가 커질수록 RMSE가 일관되게 나빠져($\alpha = 0.99$ 에서 CV 1.95), TF-IDF 예측이 semantic보다 정확하고 작은 α 가 0.5보다 낫다는 결론은 분명하다.

6. 입력 처리와 출력 포맷

`read_recommendation_input()`은 `user_id` 목록을, `read_prediction_input()`은 `user_id;game_id` 쌍을 읽는다. `semantics().tfidf()`는 user당 20줄을, `weighted()`는 쌍당 1줄을 "{},{},{:.4f}" 포맷(공백 없는 콤마, 소수점 4자리 반올림)으로 만들고, `write_output()`이 `results/`폴더를 생성해 `semantics_output.txt`, `tfidf_output.txt`, `score_prediction_output.txt`로 저장한다.

7. 어려웠던 점과 해결 방법

- **메모리:** 명세 그대로 [19755, 19755] 유사도 행렬(약 1.5 GiB)과 user × game 평점 행렬(약 103 GiB)을 만들면 메모리가 부족했다. 예측식이 유사도에 선형이라는 점을 이용해 행렬곱 순서를 바꾸고(5.2), 평점은 정렬된 테이블로 보 관했다(5.1). 캐시 없이 전체 실행에 약 74초가 걸리며, [n, 19755] 행렬을 명세 순서대로 직접 만든 참조 구현과 4-b 출력 400줄이 모두 같았다.
- **동점과 정밀도:** 이력이 1개인 user는 후보 점수가 거의 같아서, float32와 float64 중 무엇을 쓰느냐만으로 TF-IDF 추

천이 절반 이상 바뀌었다. “0.01% 차이는 결과에 영향 없음”이라는 명세와 맞도록 출력과 같은 소수점 4자리로 반올림한 뒤 정렬했다(5.2).

- **5를 넘는 semantic 점수:** semantic 추천 400줄 중 185줄이 5를 넘었고 최댓값은 906.99였다. 구현 오류를 의심했지만 원인은 음수 cosine 유사도였다. 평점 (2, 4, 3), 유사도 (−0.073, 0.043, 0.030)이면 분자 0.116, 분모 0.000128이 되어 906.99가 나온다. 가중치에 음수가 섞이면 가중 평균이 평점 범위를 벗어나고, 분모가 0에 가까우면 값이 폭발한다. 분모를 $|\text{sim}|$ 의 합으로 바꾸면 막을 수 있지만, 명세가 “유사도의 합”이고 4-b는 정답과의 일치로 평가하므로 수식을 그대로 두었다. TF-IDF는 유사도가 0 이상이라 이런 현상이 없다.
- **검증 설계와 후처리:** 전체 평점으로 만든 모델을 같은 평점으로 평가하면 정답이 예측에 섞이므로, fold마다 train 평점만으로 이력을 다시 만들었다. user의 84.6%가 평점 1개 뿐이라 검증쌍의 약 70%는 이력이 없었다. 이들을 평균 평점으로 채우고 [1, 5]로 clip하면 CV RMSE($\alpha = 0.5$)가 3.12에서 1.12로 줄었지만, 명세가 조절을 허용한 것은 α 뿐이라고 판단해 넣지 않았다. 후처리 여부에 따라 최적 α 가 0.8과 0.08로 뒤집혔기 때문에, α 는 파이프라인을 확정한 뒤 다시 골랐다.

8. 느낀 점

문장 임베딩이 tag보다 많은 정보를 담으니 예측도 더 정확할 것이라 예상했지만, 평점 예측에서는 TF-IDF가 분명히 나았다. 음수 유사도가 섞이면 가중 평균이 불안정해지는 것이 원인 중 하나로 보이며, representation의 표현력만큼 예측식과의 궁합도 중요하다는 것을 알게 되었다. 또 content 기반 방법은 user의 과거 평점이 있어야 작동하므로, 평점이 1개뿐인 user가 대부분인 데이터에서는 cold-start가 성능을 좌우했다. 수식 구현보다 동점 처리, 정밀도, 검증 분할처럼 명세에 드러나지 않은 부분을 정하는 데 더 많은 시간이 들었고, 이런 결정일수록 실험으로 확인하고 근거를 남겨야 한다고 느꼈다.